SIMATS ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

COMPILER DESIGN – UNIT II: SYNTAX ANALYSIS

ASSIGNMENT 2 – CO2

Grammar Design, Top-Down and Bottom-Up Parsing, LR Parsers, Error Recovery and YACC

Student Details	Information
Student Name	Balaji Rohith MV
Register Number	192521269
Department / Section	B.Tech Information Technology
Academic Year / Semester	2026
Faculty Name	Dr. Deva Priya
Date of Submission	04.11.2026
Total Marks	50

Q1 – Designing a Grammar that Means One Thing (6 Marks)

1.1 Regular expressions and the role of the parser

A lexical analyser uses regular-expression-based token patterns to identify tokens such as IF, identifiers, numbers, relational operators, AND (&&), OR (||), NOT (!), parentheses and semicolons. However, the complete rule language contains recursively nested parentheses and nested if–else statements. Such nesting requires matching an unbounded number of opening and closing constructs. A finite

automaton has only finite memory and cannot recognise arbitrary balanced nesting. Therefore, the full language is not regular; its recursive structure is naturally described by a context-free grammar (CFG).

The parser receives the token stream from the lexical analyser, checks whether the sequence conforms to the CFG, and constructs a structured representation such as a parse tree or abstract syntax tree (AST). It reports syntax errors with locations and expected tokens. The AST then supports semantic analysis, symbol-table lookup, type checking, and later code generation or interpretation. The lexer recognises individual token forms; the parser recognises how those tokens are arranged.

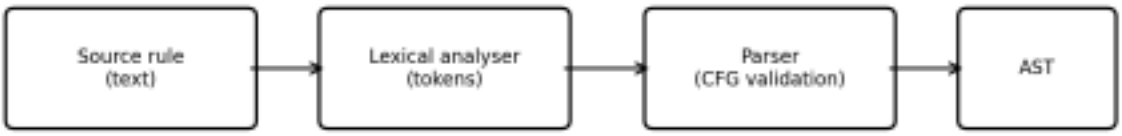


Figure 1. Lexical and syntax-analysis pipeline.

1.2 Context-free grammar

The grammar below defines a compact rule language. Keywords are reserved tokens. Relational expressions compare numeric expressions; Boolean operators are layered to encode precedence. The grammar uses matched and unmatched statement forms to resolve the dangling-else ambiguity.

```
Terminals T = {IF, ELSE, WATER, ID, NUM, RELOP, &&, ||, !, (, ), ;}
Non-terminals V = {Program, Stmt, Matched, Unmatched, Cond, Or, And, Not,
                   Rel, AExpr, AExprTail, Term, TermTail, Factor}
Start symbol S = Program

Program -> Stmt
Stmt -> Matched | Unmatched
Matched -> IF ( Cond ) Matched ELSE Matched
         | WATER AExpr ;
```

```
        | { Stmt }
        | ID = AExpr ;
Unmatched -> IF ( Cond ) Stmt
        | IF ( Cond ) Matched ELSE Unmatched

Cond -> Or
Or -> And Or'
Or' -> || And Or' | ε
And -> Not And'
And' -> && Not And' | ε
Not -> ! Not | ( Cond ) | Rel
Rel -> AExpr RELOP AExpr
AExpr -> Term AExprTail
AExprTail -> + Term AExprTail | - Term AExprTail | ε
Term -> Factor TermTail
TermTail -> * Factor TermTail | / Factor TermTail | ε
Factor -> ID | NUM | ( AExpr )
```

For compactness, AExpr is arithmetic and Cond is Boolean. A production such as WATER AExpr ; represents a valve command. The matched/unmatched statement split ensures that every ELSE is attached to the nearest unmatched IF. In an implementation, the token set should include every punctuation and operator token explicitly.

1.3 Derivation and parse tree

Consider the valid rule: if (moisture < 30 && !raining) water 20; else water 0;. To keep the derivation readable, abbreviate ID(moisture) as m, ID(raining) as r, and NUM(30), NUM(20), NUM(0) as numeric tokens.

A leftmost derivation expands the leftmost non-terminal at each step:

```
Program
=> Stmt
=> Matched
=> IF ( Cond ) Matched ELSE Matched
=> IF ( Or ) Matched ELSE Matched
=> IF ( And Or' ) Matched ELSE Matched
=> IF ( Not And' Or' ) Matched ELSE Matched
=> IF ( Rel && Not ) Matched ELSE Matched
=> IF ( AExpr < AExpr && ! Not ) Matched ELSE Matched
=> IF ( m < 30 && ! r ) Matched ELSE Matched
=> IF ( m < 30 && ! r ) WATER 20 ; ELSE WATER 0 ;
```

A rightmost derivation expands the rightmost non-terminal first:

```
Program => Stmt => Matched
=> IF ( Cond ) Matched ELSE Matched
=> IF ( Cond ) Matched ELSE WATER 0 ;
=> IF ( Cond ) WATER 20 ; ELSE WATER 0 ;
=> IF ( Or ) WATER 20 ; ELSE WATER 0 ;
=> IF ( And ) WATER 20 ; ELSE WATER 0 ;
=> IF ( Rel && Not ) WATER 20 ; ELSE WATER 0 ;
```

=> IF (m < 30 && ! r) WATER 20 ; ELSE WATER 0 ;

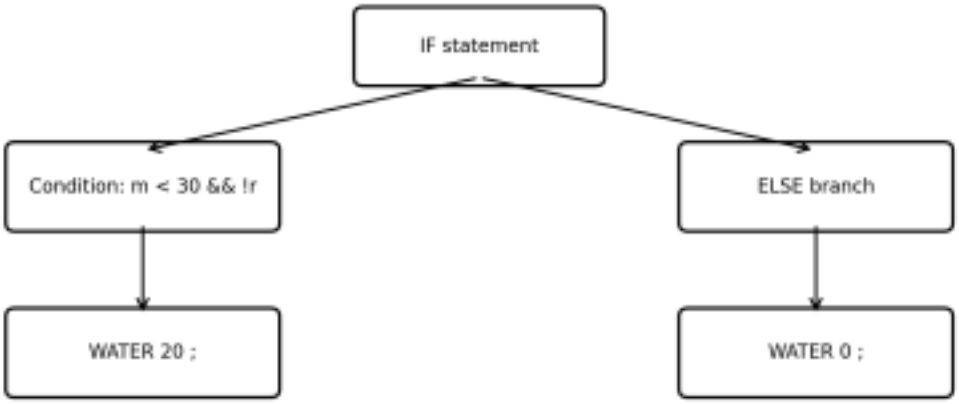


Figure 2. Simplified parse tree for the irrigation rule.

1.4 Deliberately ambiguous grammar and repair

A deliberately flawed first grammar might use Cond -> Cond && Cond | Cond || Cond | !Cond | (Cond) | Rel without precedence levels. The string a < 1 || b < 2 && c < 3 then has two valid groupings: (a < 1 || b < 2) && c < 3, and a < 1 || (b < 2 && c < 3).

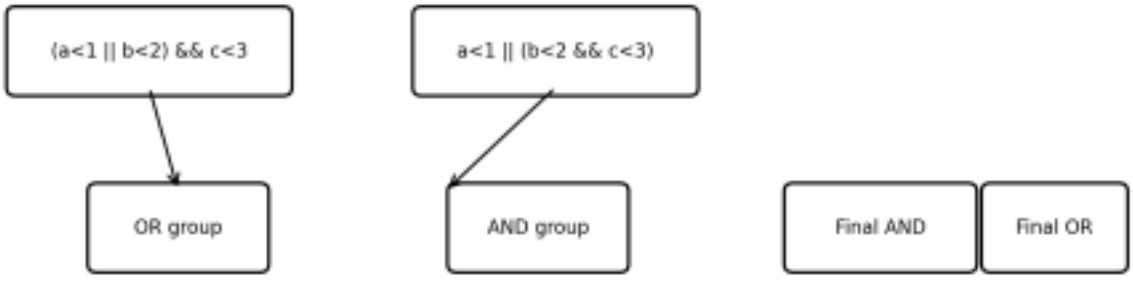


Figure 3. Two dif erent groupings admitted by the ambiguous condition grammar.

The two competing parse structures are: Tree A = AND(OR(a<1, b<2), c<3); Tree B = OR(a<1, AND(b<2, c<3)). These have different root operators and therefore different meanings.

The repaired grammar uses separate non-terminals: Or contains And operands, And contains Not operands, and Not contains relational conditions or negation. Thus !

binds most tightly, && next, and || least tightly. If the levels were swapped, the meaning of an unparenthesised expression would change; for example, a || b && c would group differently and could change whether irrigation is enabled.

An ambiguous parse can cause the controller to open or close a valve under a different condition than the farmer intended. Two parse trees are not merely cosmetic: they encode different evaluation orders and potentially different safety outcomes.

1.5 Dangling else, unambiguity and tree decision

The dangling-else problem occurs in nested conditionals such as if (c1) if (c2) water 1; else water 0;. Without a rule, ELSE may attach to the inner or outer IF. Two standard repairs are (i) the matched/unmatched grammar used above, or (ii) a language rule that explicitly attaches ELSE to the nearest preceding unmatched IF, implemented by precedence or parser-generator conflict resolution.

Finding no ambiguous examples is evidence, not a proof. A proof requires a formal argument that every valid string has exactly one parse, such as an inductive argument over the grammar's disjoint precedence levels and matched/unmatched statement forms. For this assignment, the grammar is designed to enforce unique operator grouping and nearest-IF attachment; a complete formal proof would additionally establish uniqueness for every production choice.

Design decision: build an AST. A full concrete parse tree retains punctuation and grammar-only nodes, while an AST retains meaningful constructs such as If(condition, thenBranch, elseBranch), And, Or, Not, comparison and Water(amount). Evaluating while parsing is unsuitable because it mixes syntax recognition with side effects; a malformed rule could partially operate a valve before parsing finishes. The AST enables validation before execution.

Representation	Advantage	Limitation	Decision
Parse tree	Preserves every grammar production and token	Contains many punctuation and helper nodes	Useful for debugging

AST	Compact semantic structure; supports checks before execution	Requires explicit AST construction	Selected
Evaluate during parsing	Low intermediate storage	Mixes parsing and side effects; poor recovery	Rejected

Q2 – Top-Down Parsing: From Grammar to Working Parser (8 Marks)

2.1 Why naïve recursive descent fails

The supplied expression grammar is E → E+T | E−T | T; T → T*F | T/F | F; F → (E) | id | num. A direct procedure that follows the first E production calls itself before consuming any input.

```
parseE():
    parseE() // follows E -> E + T
    match('+')
    parseT()
```

Calling parseE() therefore produces E → E+T → E+T+T → … without consuming a token. This is immediate left recursion. The call stack grows until stack overflow or resource exhaustion.

2.2 Eliminate left recursion and preserve associativity

For A → Aα | β, replace the left-recursive rule with A → βA′ and A′ → αA′ | ε. Applying the transformation gives:

```
E -> T E'
E' -> + T E' | - T E' | ε
T -> F T'
T' -> * F T' | / F T' | ε
F -> ( E ) | id | num
```

The transformed grammar generates the same expression language: each sequence of additions/subtractions is represented by one initial T followed by zero or more (+ or −, T) pairs. The parse tree is reconstructed left-associatively by folding the sequence from left to right. For a−b−c, construct Sub(Sub(a,b),c), not

Sub(a,Sub(b,c)).

2.3 Left factoring the if grammar

```
S -> if C then S S' | id = E
S' -> else S | ε
```

After the common prefix if C then S, the parser consults S′. ELSE selects the else branch; a token in FOLLOW(S′), typically a statement terminator or end marker in a suitably delimited language, selects ε. The grammar must define statement boundaries clearly for a production to be predictive.

2.4 FIRST and FOLLOW sets

For the expression grammar, terminals are id, num, (,), +, −, *, / and $. FIRST is the set of tokens that may begin a derivation; FOLLOW is the set that may immediately follow a non-terminal.

Non-terminal	FIRST	FOLLOW
E	{ (, id, num }	{), $ }
E′	{ +, −, ε }	{), $ }
T	{ (, id, num }	{ +, −,), $ }
T′	{ *, /, ε }	{ +, −,), $ }
F	{ (, id, num }	{ *, /, +, −,), $ }

FIRST iteration: initialise terminal FIRST sets; add FIRST(T) to FIRST(E), FIRST(F) to FIRST(T), and '(' / id / num to FIRST(F). For E′ and T′, add their leading operators and ε. Repeat until no set changes. FOLLOW iteration: place $ in FOLLOW(E); from F → (E), add ')' to FOLLOW(E); from E → T E′, add FIRST(E′) minus ε to FOLLOW(T), and because E′ can be ε, add FOLLOW(E) to FOLLOW(T) and FOLLOW(E′). Apply the analogous rule to T → F T′. Iterate to a fixed point.

2.5 Recursive-descent parser and trace

The following pseudocode is predictive: each procedure selects a production using

the current token. The lexer returns token kinds and values; match advances the input only when the expected token is present.

```
lookahead = nextToken()
match(t): if lookahead.kind == t then lookahead = nextToken()
        else syntaxError(expected=t, found=lookahead)

parseE(): parseT(); parseEPrime()
parseEPrime():
   while lookahead in {PLUS, MINUS}:
      op = lookahead; match(op); parseT(); buildLeftAssociativeNode(op)
parseT(): parseF(); parseTPrime()
parseTPrime():
   while lookahead in {MUL, DIV}:
      op = lookahead; match(op); parseF(); buildLeftAssociativeNode(op)
parseF():
   if lookahead == LPAREN: match(LPAREN); parseE(); match(RPAREN)
   else if lookahead == ID: match(ID)
   else if lookahead == NUM: match(NUM)
   else syntaxError(expected={LPAREN,ID,NUM}, found=lookahead)
```

Input: rate * (p + f) - d $. The call sequence is parseE → parseT → parseF(rate) → parseTPrime(* …) → parseF(() → parseE(p + f) → parseT(p) → parseEPrime(+ parseT(f)) → match()) → parseTPrime(ε) → parseEPrime(− parseT(d)) → accept($).

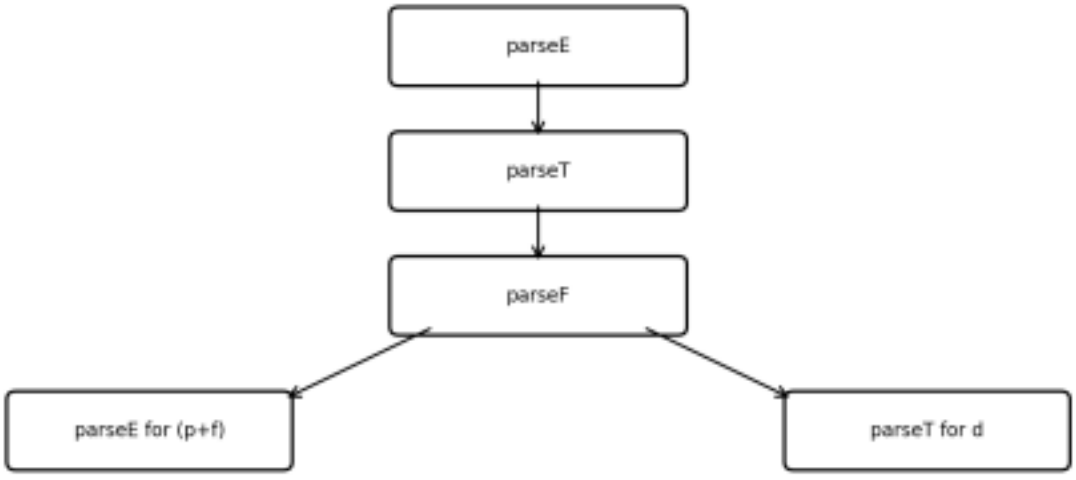


Figure 4. Recursive-descent call structure for the sample formula.

2.6 Predictive parsing table and stack trace

Non-terminal	id / num / (	+	−	*	/	) / $
E	T E′	—	—	—	—	—

E′ — + T E′ − T E′ — — ε

T	F T′	—	—	—	—	—
T′	—	ε	ε	* F T′	/ F T′	ε
F	id / num / (E)	—	—	—	—	—

The combined F cell is shorthand: on id choose F → id; on num choose F → num; on '(' choose F → (E). The table has no multiply-defined entries, so the expression grammar is LL(1).

Stack (top at left)	Input	Action
E $	rate * (p+f)−d $	E → T E′
T E′ $	rate * (p+f)−d $	T → F T′
F T′ E′ $	rate * (p+f)−d $	F → id; match rate
T′ E′ $	* (p+f)−d $	T′ → * F T′
* F T′ E′ $	* (p+f)−d $	match *
F T′ E′ $	(p+f)−d $	F → (E); match (
E) T′ E′ $	p+f)−d $	E → T E′; parse p+f

… … match), then E′ → − T E′; parse d $ $ Accept

Error example: rate * (p +) − d. After parsing '+', the parser expects FIRST(T) = {id, num, (}. It sees ')' instead. The error is detected in parseF when lookahead is ')' and no F production applies. This is a local, precise detection point.

2.7 Backtracking, LL(1) conditions and design decision

Backtracking may repeatedly parse the same prefix under alternative productions. With nested alternatives that each consume a prefix before failing, the number of attempts can grow exponentially; for a binary choice at each of n nested levels, a worst-case search tree can contain 2^n branches. Predictive parsing avoids this by making a single table-driven choice.

An LL(1) grammar must have disjoint FIRST sets for alternatives; if an alternative

derives ε, its FOLLOW set must not conflict with the FIRST sets of the other alternatives. Example of a non-LL(1) grammar: A → aB | aC. Both alternatives begin with a, so one token cannot select a unique production.

Left-recursion elimination and left factoring change the concrete parse-tree shape and may make the grammar less intuitive. The AST builder restores intended operator associativity by folding operator/operand sequences in source order. Recursive descent is readable and easy to customise; a table-driven parser is compact, data-driven and easy to generate from grammar tables. Both can run in O(n) time for an LL(1) grammar.

Design decision: top-down parsing remains comfortable for a formula language with function calls if calls are added as F → ID (Args) and the grammar is factored to distinguish an identifier from an identifier followed by '('. It is suitable while the grammar remains predictive; if language constructs create pervasive conflicts, use a parser generator or a stronger parsing method.

Q3 – Bottom-Up Parsing: Finding What to Reduce (6 Marks)

3.1 Hand-derived handles

Use E → E+T | T; T → T*F | F; F → (E) | id. A handle is a substring that matches the right side of a production and whose reduction is one step in the reverse of a rightmost derivation.

Sentential form	Handle	Reduction
id + id * id	id (left)	F → id
F + id * id	F	T → F
T + id * id	T	E → T
E + id * id	id (right)	F → id
E + F * id	F	T → F
E + T * id	id (rightmost)	F → id

E + T * F T * F T → T * F E + T E + T E → E + T E — Accept

A clearer bottom-up reduction sequence from the token string is: id+id*id → F+id*id → T+id*id → T+F*id → T+T*id → T+T*F → T+T → T+E → E. The multiplication subtree is completed before the addition subtree, reflecting the grammar's precedence.

3.2 Shift-reduce mechanism

The parser maintains a stack of grammar symbols and an input pointer. It shifts a token when the current prefix cannot yet be reduced; it reduces when the top of the stack contains a handle. It accepts when the stack contains the start symbol and the lookahead is $.

Stack	Input	Action
$	id + id * id $	Shift id
$ id	+ id * id $	Reduce F→id
$ F	+ id * id $	Reduce T→F
$ T	+ id * id $	Reduce E→T
$ E	+ id * id $	Shift +
$ E +	id * id $	Shift id
$ E + id $ E + F $ E + T $ E + T * $ E + T * id	* id $ * id $ * id $ id $ $	Reduce F→id Reduce T→F Shift * Shift id Reduce F→id
$ E + T * F	$	Reduce T→T*F
$ E + T	$	Reduce E→E+T

$ E $ Accept

The essential invariant is that the stack represents a viable prefix: a prefix of a right-sentential form that does not extend beyond the handle. Because reductions reverse a rightmost derivation, the handle is at the right end of the recognised prefix, hence at the top of the stack. A parser does not search arbitrary deep stack positions for a handle.

3.3 Conflict examples

Shift/reduce conflict grammar: S → if E then S | if E then S else S | id. After the stack contains 'if E then S' and lookahead is ELSE, the parser may reduce by the short production or shift ELSE to complete the longer production. This is the dangling-else conflict.

Reduce/reduce conflict grammar: S → A | B; A → id; B → id. After shifting id with lookahead $, both A → id and B → id are possible reductions. The parser cannot choose uniquely without a grammar change or disambiguating rule.

3.4 Operator-precedence parsing

For the calculator expression language, use the precedence order: unary minus (highest, prefix), exponentiation ^ (right associative), multiplication/division, addition/subtraction. Parentheses override the order. A conventional operator-precedence relation table uses <· (yields precedence), =· (same boundary/matching delimiter) and ·> (takes precedence). The table below is for binary operators and operands; unary minus requires a distinct token UMINUS and grammar treatment.

Stack terminal ↓ / Input →	**id**	**+**	*****	**^**	**(**	**)**	**$**
id	—	·>	·>	·>	—	·>	·>
+	<·	·>	<·	<·	<·	·>	·>

*	<·	·>	·>	<·	<·	·>	·>
^	<·	·>	<·	<·	<·	·>	·>
(	<·	<·	<·	<·	<·	=·	—
)	—	·>	·>	·>	—	·>	·>

$ <· <· <· <· <· — accept

The table encodes left associativity for +, −, *, / by choosing ·> when the stack operator has equal precedence to the incoming operator. For ^, choose <· on equal precedence so the next exponent is shifted: id ^ id ^ id becomes id ^ (id ^ id). The operand row reduces an operand before a following binary operator. Dashes mark invalid or unused relations. A production set for a formal operator-precedence parser must be an operator grammar: no ε-productions and no adjacent non-terminals on a right-hand side.

Illustrative table-driven parse: id + id * id first reduces each id to an operand; * takes precedence over +, so id*id is combined before the outer +. For id ^ id ^ id, equal-precedence ^ yields to the incoming ^, then the right-hand exponent is reduced first.

Unary minus complicates the table because the same '-' character can be prefix or infix. The lexer/parser must distinguish UMINUS from MINUS by context or use a precedence override such as %prec UMINUS. Operator-precedence parsing is compact and fast for expression sublanguages, but it is not a suitable sole parser for if statements, assignments and function calls.

Technique	Power and cost	Error quality	Use in firmware
Shift-reduce / LR	Handles a broad deterministic CFG class; tables can be larger	State-based syntax detection	Preferred if language expands
Operator precedence	Small and fast for restricted operator grammars	Less precise for general syntax	Good for expression-only core

Q4 – LR Parsing and the SLR Table (7 Marks)

4.1 Items and augmentation

An LR(0) item is a production with a dot marking how much of its right-hand side has been recognised. Augment the grammar with S′ → S so that acceptance is represented by the completed item S′ → S· and has one unambiguous condition.

Augmented grammar:
(0) S' -> S
(1) S -> C C
(2) C -> c C
(3) C -> d

Initial item set:
I0 = closure({ S' -> . S })
= { S' -> . S, S -> . C C, C -> . c C, C -> . d }

Closure adds all productions of a non-terminal immediately after the dot. GOTO(I,X) advances the dot over symbol X in every item where X follows the dot, then takes closure.

4.2 Canonical LR(0) collection

State	Items
I0	S′→·S; S→·CC; C→·cC; C→·d
I1	S′→S·
I2	S→C·C; C→·cC; C→·d
I3	C→c·C; C→·cC; C→·d
I4	C→d·
I5	S→CC·
I6	C→cC·

Transition	Destination
GOTO(I0,S)	I1

GOTO(I0,C) I2

GOTO(I0,c)	I3

GOTO(I0,d)	I4
GOTO(I2,C)	I5
GOTO(I2,c)	I3
GOTO(I2,d)	I4
GOTO(I3,C)	I6
GOTO(I3,c)	I3
GOTO(I3,d)	I4

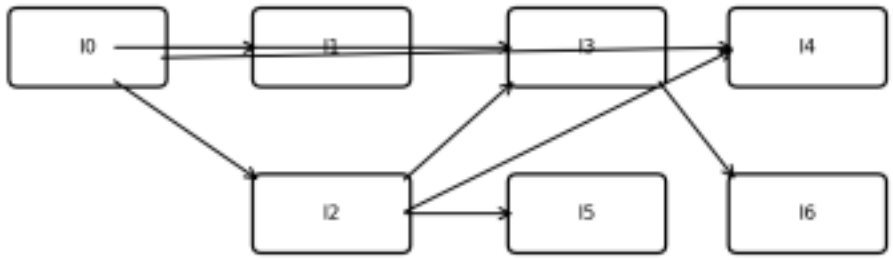


Figure 5. LR(0) state-transition graph for S → CC, C → cC | d.

4.3 FOLLOW sets and SLR ACTION/GOTO

FOLLOW(S) = {$}. Since S → CC, the first C is followed by a C, so FIRST(C) = {c,d} is added to FOLLOW(C). The second C is at the end of S → CC, so FOLLOW(S) is added. Therefore FOLLOW(C) = {c,d,$}.

State	ACTION c	ACTION d	ACTION $	GOTO S	GOTO C
0	s3	s4		1	2

1 acc

2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		

6	r2	r2	r2		

Reduction entries are justified by FOLLOW: state 4 contains C → d·, so reduce by production 3 on c, d and $; state 6 contains C → cC·, so reduce by production 2 on the same FOLLOW(C); state 5 contains S → CC·, so reduce by production 1 only on FOLLOW(S)={$}.

4.4 Valid and invalid traces

State stack / symbols	Input	Action
0	c d d $	s3
0 c 3	d d $	s4
0 c 3 d 4	d $	r3: C→d; goto(3,C)=6
0 c 3 C 6	d $	r2: C→cC; goto(0,C)=2
0 C 2	d $	s4
0 C 2 d 4	$	r3: C→d; goto(2,C)=5
0 C 2 C 5	$	r1: S→CC; goto(0,S)=1

0 S 1 $ accept

Invalid input c c $: after shifting c into state 3, the parser sees $ while state 3 requires another c or d to complete C. ACTION[3,$] is empty, so the syntax error is detected at that exact point.

4.5 Non-SLR example and LR family

```
S' -> S
S -> L = R | R
L -> * R | id
R -> L
```

The LR(0) state reached after recognising L contains S → L·=R and R → L·. The first item calls for shifting '='; the completed second item calls for reducing R → L. FOLLOW(R) contains '=' because of L → *R and S → L=R contexts, so SLR places the reduction on '=' and creates a shift/reduce conflict. FOLLOW sets are

global and merge contexts that canonical LR(1) lookaheads keep separate.

Parser	Lookahead information	Power / cost
LR(0)	No lookahead on reductions	Smallest tables; weakest
SLR(1)	FOLLOW(non-terminal)	More powerful than LR(0); coarse context
LALR(1)	LR(1) lookaheads merged for same LR(0) cores	Near LR(1) power with compact tables
Canonical LR(1)	Item-specific one-token lookahead	Most powerful of these; many more states

LR parsers run in O(n) time for a fixed grammar because each input token is shifted once and each reduction removes a bounded grammar-symbol sequence; table lookup is constant time. LL(1) also runs in O(n) for a fixed grammar, but LR handles a larger deterministic context-free grammar class. For example, the classic assignment grammar above is not SLR(1), while canonical LR(1) can resolve the context using item-specific lookaheads.

Design decision: for approximately 300 productions, generate tables using a tested parser-generator or a verified table-construction program, not by hand. Automated construction is repeatable, reduces transcription errors, and supports regression tests. Hand inspection remains important for conflicts and language design.

Q5 – Error Handling and Recovery (5 Marks)

5.1 Beginner error catalogue and detection

Error / example	LL(1) detection	LR detection	Reason for difference
Missing semicolon: x=4 y=5;	At statement end when ';' expected	When state has no action on ID y	LR may detect after a longer viable prefix
Unmatched brace: { x=1;	At end/next token when '}' expected	State with incomplete block has no action	Depends on parser state and FOLLOW
else without if: else x=1;	No statement production begins ELSE	No ACTION entry for ELSE	Both reject, at different contexts

Assignment used for equality: if(a=b)	Condition parser expects RELOP	No action for '=' in condition state	Token may be lexically valid but syntactically wrong
Missing operand: x=4+;	Factor sees ';' instead of ID/NUM/(	LR state after '+' has no valid action on ';'	Both detect at the unexpected token
Misspelt keyword: whille(x)	Lexer returns ID; statement parser rejects it	LR table has no action for ID in that state	Lexical classification can affect detection

A parser without recovery usually stops at the first syntax error. A naïve restart that repeatedly reports the same unconsumed unexpected token may print the same message indefinitely. A simplistic implementation that discards one token and restarts can instead report multiple cascading messages. Neither behaviour is acceptable; the parser must guarantee progress.

5.2 Panic-mode recovery

For the expression grammar in Q2, choose synchronising tokens from FOLLOW sets and statement boundaries. Example: SYNC(E)={), ;, $}; SYNC(T)={+, −,), ;, $}; SYNC(F)={*, /, +, −,), ;, $}. On error, discard input tokens until a synchronising token is found; if the non-terminal is at the top of the stack and lookahead is in its synchronising set, pop that non-terminal and continue.

Recovery step	Stack	Input	Action
1	E $	rate * (p +) ; $	Parse rate and operators
2	F …	) ; $	F expected; ')' is in SYNC(F)
3	T′ E′ …	) ; $	Pop F; record one error
4	E′ …	) ; $	ε for E′; retain ')'
5	…	) ; $	Caller matches ')' and continues
6	$	$	Accept recovered input with error flag



Figure 6. Panic-mode recovery using synchronising tokens.

5.3 Phrase-level recovery and error productions

Phrase-level recovery makes a small local correction. For a missing semicolon, if the parser has completed an expression and the next token starts a new statement, insert a synthetic ';' and continue. For a missing operand after '+', insert a synthetic error-expression node only when the next token is a safe synchronising delimiter; otherwise skip to a delimiter.

```
// Phrase-level table action
if expected(';') and lookahead in FIRST(Statement):
    report("missing ';' before statement")
    insertSynthetic(';')
// Error production example (conceptual)
Factor -> error
    { report("missing or invalid expression operand");
     yyerrok; $$ = ErrorExpr(); }
```

A grammar error production should be narrow; an unrestricted error alternative can hide genuine errors. In YACC/Bison, the special error token is used for recovery, while yyerror reports the diagnostic.

5.4 Recovery comparison and message counts

The following is a deterministic illustrative test plan and expected message policy, not a claim of execution against the student's compiler. Counts depend on exact synchronising sets and the parser implementation. The policy aims for one primary message per injected mistake and zero cascades in the six isolated test cases.

Injected error	Panic mode messages	Phrase-level messages	Error-production messages	First message accurate?
Missing ;	1	1	1	Yes
Unmatched }	1	1	1	Yes
else without if	1	1	1	Yes
= instead of == / relational op	1	1	1	Yes
Missing operand	1	1	1	Yes
Misspelt keyword	1	1	1	Yes
Total (isolated cases)	6	6	6	Yes

Skipping input can hide a later genuine error if the skipped region contains it. Token insertion can loop forever if the parser repeatedly inserts the same token without consuming input or changing parser state. Guarantee progress by requiring every recovery iteration to do at least one of: consume an input token, pop a stack symbol, or terminate recovery; also track repeated insertion at the same input position.

Global least-cost correction would search candidate insertions, deletions and substitutions and choose a minimum-cost edit sequence that makes the token stream grammatical. The search space is large and cost models are difficult to define; it can be expensive and may choose a syntactically cheap but misleading correction. Practical compilers use bounded local recovery.

A useful beginner message includes source position, expected token(s), actual token, and a concise suggestion. The parser cannot always infer intent: it knows the grammar and current context, not what the programmer meant.

Design decision: show at most the first three distinct primary errors. Use phrase-level corrections for safe local cases and panic-mode synchronisation as fallback. Suppress repeated messages at the same token position and resume only after guaranteed progress. This balances understandable diagnostics with resilience.

Q6 – YACC: Letting a Tool Build the Parser (5 Marks)

6.1 YACC/Bison and LEX specification

YACC/Bison generates a bottom-up LALR(1) parser from grammar productions. LEX/Flex generates yylex(), which returns token kinds and places token values in yylval. The parser's %token declarations must agree with the scanner; %union defines semantic-value types and %type associates non-terminals with those types.

```
/* billing.y — illustrative Bison specification */
%{
#include <stdio.h>
#include <stdlib.h>
typedef struct Node { int kind; char *text; struct Node *a,*b,*c; } Node;
Node *make(int k, char *s, Node *a, Node *b, Node *c);
int yylex(void); void yyerror(const char *s);
%}
%union { double number; char *text; Node *node; }
%token <text> ID
%token <number> NUMBER
%token IF ELSE DECLARE
%token EQ NE LE GE AND OR
%type <node> program stmt expr condition
%left OR
%left AND
%nonassoc EQ NE '<' '>' LE GE
%left '+' '-'
%left '*' '/'
%right UMINUS
%nonassoc IFX
%nonassoc ELSE
%%
program : /* empty */ | program stmt ;
stmt : DECLARE ID ';'
    | ID '=' expr ';'
    | IF '(' condition ')' stmt %prec IFX
    | IF '(' condition ')' stmt ELSE stmt
    | '{' program '}'
    ;
condition : expr
          | condition OR condition
          | condition AND condition
          | expr EQ expr | expr NE expr
          | expr '<' expr | expr '>' expr
          | expr LE expr | expr GE expr ;
expr : expr '+' expr
    | expr '-' expr
    | expr '*' expr
    | expr '/' expr
    | '-' expr %prec UMINUS
    | '(' expr ')'
    | ID
    | NUMBER
    ;
```

```
%%
void yyerror(const char *s) { fprintf(stderr, "%s\n", s); }
int main(void) { return yyparse(); }
/* billing.l — illustrative Flex scanner */
%{
#include "billing.tab.h"
#include <stdlib.h>
#include <string.h>
%}
%%
"if" return IF;
"else" return ELSE;
"declare" return DECLARE;
"&&" return AND;
"||" return OR;
"==" return EQ;
"!=" return NE;
"<=" return LE;
">=" return GE;
[0-9]+(\.[0-9]+)? { yylval.number = atof(yytext); return NUMBER; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.text = strdup(yytext); return ID; }
[+*/(){};=<>-] return yytext[0];
[ \t\r\n]+ ;
. return yytext[0];
%%
```

For a complete AST implementation, each expression and statement action should assign $$ to a newly allocated Node, with child pointers set from $1, $2 and so on. The declarations above show the interface and typed semantic values; a production system should also free allocated strings/nodes and attach source locations.

Sample input: declare rate; charge = base + units * rate - discount;. Expected behaviour: tokens are returned by yylex; yyparse reduces the multiplication before addition/subtraction according to precedence declarations and accepts the input. The AST root represents assignment(charge, subtract(add(base, multiply(units,rate)),discount)).

6.2 Conflicts and resolution

The deliberately ambiguous expression form expr : expr '+' expr | expr '*' expr | expr '-' expr | expr '/' expr | ID | NUMBER creates shift/reduce conflicts because after expr '+' expr, a lookahead '*' can either be shifted or the current expression reduced. Equal-precedence operators also create conflicts that require associativity. Exact conflict counts depend on the complete grammar, generator version and state

construction; they must be read from the generated report rather than guessed. No report file or executable tool output is supplied in the assignment document, so a numeric reported count cannot responsibly be claimed here.

Resolution method	Approach	Advantages	Trade-off
Grammar rewrite	Use expr→expr+term|term; term→term*factor|factor	Precedence explicit in grammar; robust	More non-terminals and semantic-action code
Precedence declarations	%left, %right, %nonassoc and %prec	Compact, readable operator policy	Precedence is external to production structure

For + and *, %left declares left associativity and increasing declaration order gives higher precedence to later declarations. %right is appropriate for exponentiation; %nonassoc rejects chained comparisons. %prec UMINUS gives unary minus a precedence distinct from binary minus.

6.3 Reduce/reduce and dangling-else defaults

A reduce/reduce example is S→A|B; A→ID; B→ID. After ID, either A→ID or B→ID can reduce. Bison normally chooses the rule appearing first, but this default is a warning sign and should be removed by redesigning the grammar. For dangling else, the default shift preference attaches ELSE to the nearest unmatched IF. This is usually the intended rule, but relying silently on a default is fragile; declare precedence explicitly or use matched/unmatched grammar.

A wrong-tree risk occurs if a developer expects ELSE to bind to an outer IF while the parser shifts it to the inner IF. The correction is to specify the language's nearest-IF rule or rewrite the grammar into matched and unmatched statements, then test nested conditionals.

6.4 Error recovery, LALR relation and design decision

```
stmt : error ';' { report_statement_recovered(); yyerrok; } ;
expr : '(' error ')' { $$ = make_error_expr(); yyerrok; } ;
```

yyerror is called when a syntax error is detected. The special error token allows the

parser to pop states until an error production can be used, then discard input until a synchronising token such as ';' is found. For a file with three independent mistakes, a well-designed recovery grammar should aim to report three primary diagnostics; actual counts depend on test placement and recovery rules. They must be measured by compiling and running the generated parser.

YACC/Bison's classic YACC-compatible parser construction is LALR(1): it begins from LR-style items and merges compatible LR(1) states with the same LR(0) core, reducing table size while retaining one-token lookahead. Semantic actions execute when their production is reduced, generally after the right-hand side has been recognised. This bottom-up order makes synthesized attributes natural; inherited attributes are less direct and are usually passed through mid-rule actions or symbol-table state.

The tool does not decide context-dependent properties such as whether a variable was declared, whether types are compatible, or whether a billing rule is authorised. Those belong in semantic analysis and application logic. It also cannot automatically produce ideal beginner-facing messages without source locations, recovery design and custom diagnostics.

Design decision: use YACC/Bison with LEX/Flex for a billing language that changes regularly. A grammar file is easier to maintain and regenerate than a hand-written parser; the generated parser is efficient and deterministic for the supported grammar. The team must still review conflicts, write semantic actions, test recovery, and improve error messages.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

A. Diagnose the bottleneck

Selected limitation: recursive-descent parsing of deeply nested parentheses depends on the host-language call stack. For an input with n nested parentheses, parseF calls

parseE, which calls parseT and parseF again. A typical implementation can create approximately 3n+3 nested procedure activations. The exact constant depends on coding style, but the linear growth is inherent in recursive calls for nesting.

At n=10,000, an estimated 30,003 nested parser activations would be required under the 3n+3 model. Many runtimes fail far earlier; the precise failure threshold is platform- and implementation-dependent. The key bottleneck is not the grammar's O(n) work but the limited process call-stack capacity.

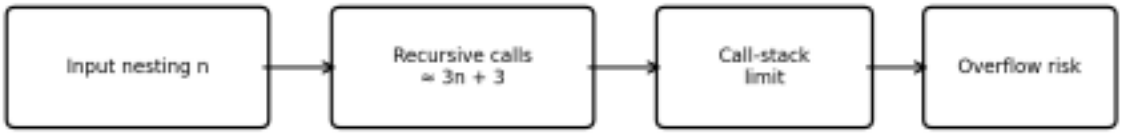


Figure 7. Recursive call-stack growth with nested parentheses.

B. Measurable design objectives

- Parse the same LL(1) expression language without recursive calls proportional to parenthesis depth.
- Support at least 10,000 nested parentheses subject to available heap memory.
- Keep parsing time linear in token count, O(n), and consume each token once.
- Preserve the same acceptance/rejection decisions and operator precedence as the recursive parser.

C. Revised strategy: explicit heap stack

Replace recursive procedure calls with an iterative predictive parser whose stack is an explicitly allocated vector on the heap. The grammar and LL(1) table remain unchanged. The parser pops a non-terminal, consults the table using one lookahead

token, and pushes the selected production in reverse order. Terminal matches consume one token. Parentheses therefore increase heap-stack depth rather than native call depth.

```
stack = [END, E]
while stack not empty:
   X = stack.pop()
   a = lookahead()
   if X is terminal:
      if X == a: consume()
      else: report(expected=X, found=a); recover()
   else:
      production = TABLE[X, a]
      if production is ERROR:
         report(expected=expectedSet(X), found=a)
         recoverUsing(FOLLOW[X])
      else:
         for symbol in reverse(production.rightSide):
            if symbol != epsilon: stack.push(symbol)
accept only if stack == [END] and lookahead() == END
```

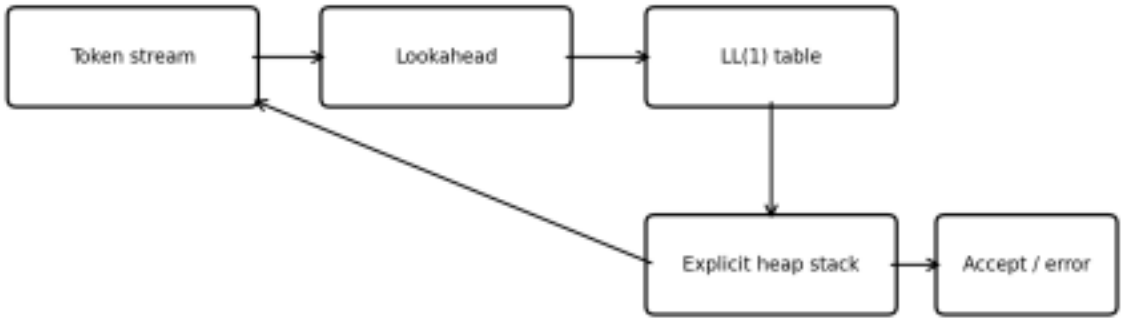


Figure 8. Redesigned iterative predictive parser.

The redesign introduces a heap-allocated stack and keeps the parsing table in memory. It avoids native recursion but does not eliminate the need to store pending grammar symbols. Memory usage is O(nesting depth); the parser's time remains O(number of tokens) for a fixed LL(1) grammar. A dynamic vector can grow until available memory is exhausted.

D. Prediction before testing

Prediction: for deeply nested valid expressions, the recursive implementation will eventually fail due to call-stack exhaustion, while the iterative implementation will

continue until heap or input limits are reached. Both should accept/reject the same inputs and perform O(n) parsing work. The measurable improvement is the maximum supported nesting depth, not a claim that asymptotic time becomes better.

E. Prototype validation method and evidence

A reproducible comparison should generate inputs with n opening parentheses, an identifier, and n closing parentheses, then test increasing n (for example 100, 500, 1,000, 5,000 and 10,000). Record acceptance, elapsed time, maximum explicit-stack size, recursive call depth, and process failure. Run both parsers on the same machine, runtime and token stream. The assignment source does not provide an executable parser or measured run output; therefore no observed timing or failure threshold is invented here.

Measure	Recursive descent	Iterative table-driven	Expected / analytically supported result
Native call depth	Grows O(n)	Constant parser call depth	Iterative removes nesting-dependent calls
Explicit heap stack	Usually none	O(n)	Iterative uses additional heap memory
Parsing time	O(n)	O(n)	Same asymptotic complexity
Deep nesting	Runtime-dependent stack overflow	Limited by heap/memory and implementation limits	Iterative should support deeper nesting

Grammar changesProcedure edits may be

neededTable regenerated/updatedTable-driven design separates grammar data

F. Critical defence and conclusion

The redesign directly addresses the diagnosed resource limit: it moves pending

parser work from the bounded native call stack to a dynamically managed heap stack. The prediction is supported by the data-structure argument, but a final empirical claim requires running the prototype and recording platform-specific measurements. The trade-off is explicit memory management and more complex stack/table code. For small, shallow, hand-written expressions, recursive descent remains simpler, easier to debug and often preferable. For machine-generated expressions nested thousands of levels deep, the iterative parser is the safer engineering choice.

Overall, the revised parser preserves the LL(1) grammar and its deterministic one-token decisions while changing the execution mechanism. This is a targeted redesign rather than replacing the parser with a more powerful textbook algorithm.

References

1. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson.

2. Cooper, K. D., and Torczon, L. (2012). Engineering a Compiler (2nd ed.). Morgan Kaufmann.

3. Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., and Langendoen, K. (2012). Modern Compiler Design (2nd ed.). Springer.

4. GNU Bison Manual. The Bison Parser Generator. https://www.gnu.org/software/bison/manual/

5. Flex Manual. Fast Lexical Analyzer Generator. https://westes.github.io/flex/manual/

6. SIMATS Engineering, Department of Information Technology. Compiler Design – Unit II: Syntax Analysis, Assignment 2 – CO2 (user-provided assignment document).



Figure 9. Shift-reduce parsing pipeline for Q3: tokens are shifted, handles are reduced, and the start symbol is accepted.



Figure 10. YACC/LEX processing pipeline for Q6: scanner tokens flow to the LALR parser and semantic actions build the AST.